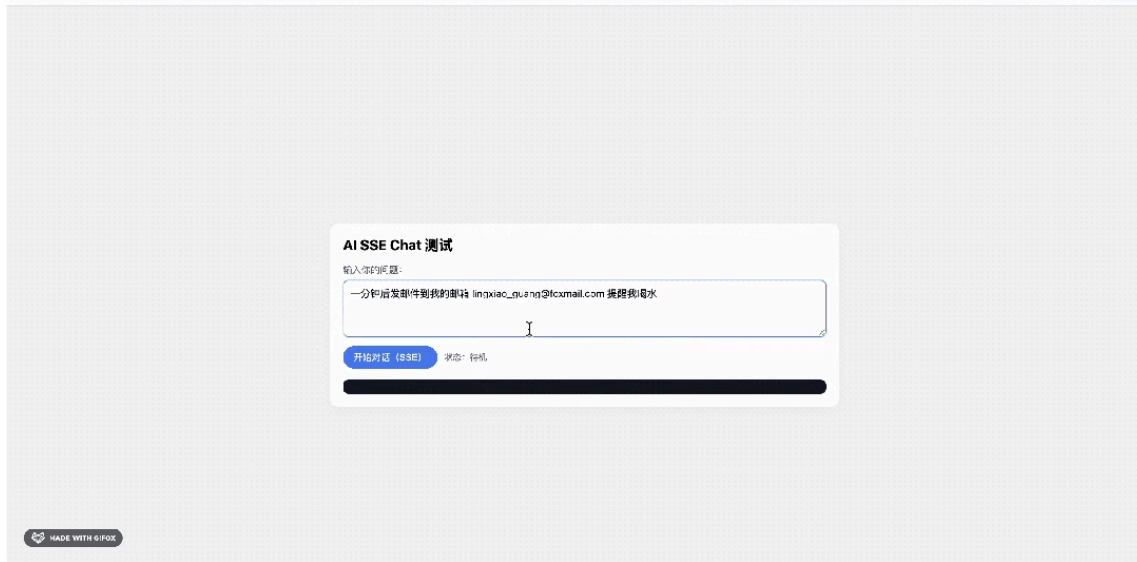


AGUI 协议：Vercel AI SDK + LangChain 实现流式组件渲染 已付费

原创 神说要有光 神光的幸福生活 2026年4月4日 19:04 山东

我们前面做的 Agent 功能上没啥问题，但是 UI 比较简陋，只有流式的文字：



而你用 cursor 之类的 Agent，它的界面是这样的：

Grepped appState\.\lm\.\lm\b in src

Executing build process 2s

完成主要代码替换后，我会运行一次 `npm run build`（包含 `vue` 查）来确认删除 ASR/LLM 文件以及 `appState` 结构调整后，整个编译。若有 TS/模板报错，我会再针对性修复。

 Ran command `cd, npm run`

```
$ cd "/Users/guang/Downloads/基础接入demo" && npm
build
```

```
demo1@0.0.0 build
vue-tsc -b && vite build
```

```
rc/stores/app.ts(2,25): error TS6196: 'AvatarCo
rc/stores/app.ts(61,13): error TS6133: 'start'
```

^

Explored 1 file

TS app.ts +218 -219

```
import { reactive, ref } from 'vue'
import type { AppState, AvatarConfig } from '../typ
import { APP_CONFIG } from '../constants'
```

除了流式的文字，不同的 tool call 有不同的组件来展示，这样体验就好很多。

这种流式返回文字，还能流式渲染组件，需要一套协议。

叫做 AGUI 协议（Agent–User Interaction Protocol），定义 agent 和图形界面怎么交互的

比如我们之前返回的 SSE 消息是这样的：

只有文字，并不能区分是文本内容，还是 tool call，需要一些元信息，比如 type。

解决也很简单，返回 json 就好了。

比如这样：

text-start 代表文本流开始

text-delta 是流式的文本数据

text-end 代表文本流结束

如果有 tool call 就是这样：

tool-input-start 代表开始接收到 tool 的参数

tool-input-delta 是流式的 tool call 的参数

tool-input-available 代表 tool 的参数接收完

tool-output-available 代表有了 tool 的调用结果，可以从 output 里取

这样 SSE 不止返回流式文本，而是这种 json，那前端不就知道当前是在工具调用还是输出流式文本了么？

自然就可以渲染不同的组件，实现更好的体验。

上面的是 Vercel AI SDK 实现的协议，我们直接用它那个就行。

<https://ai-sdk.dev/docs/ai-sdk-ui/stream-protocol#data-stream-protocol>

在 Vercel AI SDK 里叫做 Data Stream Protocol

Vercel AI SDK 提供了这些包：

- **ai 包**：写 agent 逻辑
- **@ai-sdk/openai、@ai-sdk/anthropic 等包**：对接不同的大模型，就和 langchain 的 ChatModel 一样
- **@ai-sdk/react、@ai-sdk/vue 等包**：对接后端接口，实现页面渲染

用它也可以写 Agent，但它功能比较少，我们只用它的 UI 方面的功能，就是刚才的那套 AGUI 的协议。

它提供了和 LangChain 的集成包 @ai-sdk/langchain

我们用 LangChain 写 Agent 部分，然后复用它这套 AGUI 协议来给前端传输消息

前端用 @ai-sdk/react、@ai-sdk/vue 等来解析 SSE 的消息，拿到 messages，用不同组件渲染就可以了。

我们来创建个后端项目：

```
nest new agui-backend
```

安装用到的包：

```
pnpm install @langchain/core @langchain/openai @nestjs/config zod
```

创建 .env 配置文件：


```
OPENAI_API_KEY=sk-xx
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
MODEL_NAME=qwen-plus

BOCHA_API_KEY=sk-xx
```

然后引入 ConfigModule 读取配置：

创建 ai 模块：

```
nest g module ai
nest g controller ai --no-spec
nest g service ai --no-spec
```

然后来写个 SSE 的 ai 接口：

改下 AiModule，加一下网络搜索的 tool

```
import { Module } from '@nestjs/common';
import { AiService } from '../ai.service';
import { AiController } from '../ai.controller';
import { ConfigService } from '@nestjs/config';
import { ChatOpenAI } from '@langchain/openai';
import { tool } from '@langchain/core/tools';
import z from 'zod';

@Module({
  controllers: [AiController],
  providers: [AiService,
    {
      provide: 'CHAT_MODEL',
      useFactory: (configService: ConfigService) => {
        return new ChatOpenAI({
          model: configService.get('MODEL_NAME'),
          apiKey: configService.get('OPENAI_API_KEY'),
          configuration: {
            baseUrl: configService.get('OPENAI_BASE_URL'),
          },
        });
      },
    },
    {
      provide: 'WEB_SEARCH_TOOL',
      useFactory: (configService: ConfigService) => {
```

```

const webSearchArgsSchema = z.object({
  query: z
    .string()
    .min(1)
    .describe('搜索关键词，例如：公司年报、某个事件等'),
  count: z
    .number()
    .int()
    .min(1)
    .max(20)
    .optional()
    .describe('返回的搜索结果数量，默认 10 条'),
});

return tool(
  async ({ query, count }: { query: string; count?: number }) => {
    const apiKey = configService.get<string>('BOCHA_API_KEY');
    if (!apiKey) {
      return 'Bocha Web Search 的 API Key 未配置（环境变量 BOCHA_API_KEY），请先在服务端配置';
    }

    const url = 'https://api.bochaai.com/v1/web-search';
    const body = {
      query,
      freshness: 'noLimit',
      summary: true,
      count: count ?? 10,
    };

    const response = await fetch(url, {
      method: 'POST',
      headers: {
        Authorization: `Bearer ${apiKey}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(body),
    });

    if (!response.ok) {
      const errorText = await response.text();
      return `搜索 API 请求失败，状态码：${response.status}，错误信息：${errorText}`;
    }

    let json: any;
    try {
      json = await response.json();
    } catch (e) {

```

```

        return `搜索 API 请求失败, 原因是: 搜索结果解析失败 ${e as Error}.message`;
    }

    try {
        if (json.code !== 200 || !json.data) {
            return `搜索 API 请求失败, 原因是: ${json.msg ?? '未知错误'}`;
        }

        const webpages = json.data.webPages?.value ?? [];
        if (!webpages.length) {
            return '未找到相关结果。';
        }

        const formatted = webpages
            .map(
                (page: any, idx: number) =>
                    `引用: ${idx + 1}
标题: ${page.name}
URL: ${page.url}
摘要: ${page.summary}
网站名称: ${page.siteName}
网站图标: ${page.siteIcon}
发布时间: ${page.dateLastCrawled}`
            )
            .join('\n\n');

        return formatted;
    } catch (e) {
        return `搜索 API 请求失败, 原因是: 搜索结果解析失败 ${e as Error}.message`;
    }
},
{
    name: 'web_search',
    description:
        '使用 Bocha Web Search API 搜索互联网网页。输入为搜索关键词（可选 count 指定结果数量',
    schema: webSearchArgsSchema,
},
);
},
inject: [ConfigService],
},
],
})
export class AiModule {}

```

这里创建了 ChatModel 和网络搜索的 tool 的 provider

然后在 AiService 注入：

```
import { Inject, Injectable } from '@nestjsjs/common';
import { ChatOpenAI } from '@langchain/openai';
import { AIMessage, AIMessageChunk, createAgent, HumanMessage, SystemMessage, ToolMessage }
import { UIMessage } from 'ai';
import { toBaseMessages, toUIMessageStream } from '@ai-sdk/langchain';

@Injectable()
export class AiService {
  private readonly agent: ReturnType<typeof createAgent>;

  constructor(
    @Inject('WEB_SEARCH_TOOL') private readonly webSearchTool: any,
    @Inject('CHAT_MODEL') model: ChatOpenAI
  ) {
    this.agent = createAgent({
      model,
      tools: [this.webSearchTool],
      systemPrompt:
        '你是 AI 助手，需要最新信息、事实核查或联网信息时，请使用 web_search 工具搜索后再作答。',
    });
  }

  async stream(messages: UIMessage[]) {
    const lcMessages = await toBaseMessages(messages);
    const lgStream = await this.agent.stream(
      { messages: lcMessages },
      {
        streamMode: ['messages', 'values'],
        recursionLimit: 12,
      },
    );

    return toUIMessageStream(lgStream as AsyncIterable<AIMessageChunk>);
  }
}
```

这次我们不再手写 agent loop，自己调用 tool 了，直接用 langchain 封装好的 createAgent 的 api

然后用 @ai-sdk/langchain 这个适配器：

把传入的 ai sdk 的 messages 转成 langchain 的 BaseMessage 传给 agent

再把返回的 stream 转成 ai ask 的 ui message stream 返回

这样返回的流式内容就是 SSE 的 Data Stream Protocol 的协议数据了。

我们改下 AiController，加一下接口：

```
import { BadRequestException, Body, Controller, Get, Post, Query, Res, Sse } from '@nestjs'
import type { Response } from 'express';
import { AiService } from './ai.service';
import { pipeUIMessageStreamToResponse, UIMessage } from 'ai';

@Controller('ai')
export class AiController {
  constructor(private readonly aiService: AiService) {}

  /**
   * 本地测试:
   * curl -N -sS -X POST 'http://localhost:3000/ai/chat' \
   *   -H 'Content-Type: application/json' \
   *   -d '{"messages":[{"id":"1","role":"user","parts":[{"type":"text","text":"北京今天的天
   */
  @Post('chat')
  async postChat(
    @Body() body: { messages?: UIMessage[] },
    @Res({ passthrough: false }) res: Response,
  ): Promise<void> {
    if (!body?.messages || !Array.isArray(body.messages)) {
      throw new BadRequestException('Invalid JSON');
    }

    const stream = await this.aiService.stream(body.messages);
    pipeUIMessageStreamToResponse({ response: res, stream });
  }
}
```

因为 ai sdk 转换好的就是 SSE 的流，我们不需要自己再做处理，直接把它传给 response 就可以了。

安装用到的 ai sdk 的包：

```
pnpm install ai @ai-sdk/langchain
```

用上面那个 curl 测试下：

神光的编程秘籍

对接成功！

现在就把 langchain 的 agent 的 stream 转成了 ai sdk 的 Data Stream Protocol 协议的格式了。

接下来创建前端项目：

```
npx create-vite agui-frontend
```

这里创建的是 react 项目，用 @ai-sdk/react 来对接，你换成 vue 项目，用 @ai-sdk/vue 对接也可以。

vercel ai sdk 支持各种前端框架

在后端允许下跨域访问接口：

然后来改前端页面：

安装 @ai-sdk/react 和 ai 包


```
pnpm install @ai-sdk/react ai
```

核心逻辑是这个：

用 `useChat` 连接后端的 SSE 接口，连接方式用 `DefaultChatTransport`

这样就可以拿到 `messages` 了，不用自己解析

`message` 有 `id`、`role`、`parts` 属性：

创建个组件渲染 `parts` 部分：

ai 包提供了 isToolUIPart、getToolName 的 api

我们可以用它来判断当前 part 是不是 tool call

如果不是，就是渲染文本，如果是就是渲染对应的 tool 的组件。

用 `getToolName` 拿到 `part` 的工具名

目前只有 `web search` 的 `tool`，根据 `state` 来渲染 `pending`、`error` 状态的组件，还有成功后的组件

就像前面分析的，`output-available` 阶段可以拿到 `output`：

根据不同 `tool` 的 `output` 的格式做下渲染就可以了。

具体代码可以从仓库复制，核心的就是刚才讲的这几个，其余的不重要。

跑一下：

```
npm run dev
```

神光的编程秘籍

现在就不只是流式渲染文本了，还会流式渲染 tool call 对应的组件

但现在流式文本部分的 markdown 还没处理：

豆包里是这样的：

神光的编程秘籍

我们加一个流式渲染 markdown 对应组件的库 Streamdown

安装依赖：

```
pnpm install streamdown @streamdown/code @streamdown/mermaid
```

这样流式的 markdown 文本就会用对应组件来渲染了：

我们只做了 web search 的 tool，再来加一个 tool

把之前发送邮件的 tool 拿过来

安装下依赖：

```
pnpm install @nestjs-modules/mailer
```

在 .env 加对应配置：

在 AppModule 引入这个包：

```
MailerModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (configService: ConfigService) => ({
    transport: {
      host: configService.get<string>('MAIL_HOST'),
      port: Number(configService.get<string>('MAIL_PORT')),
```



```

    secure: configService.get<string>('MAIL_SECURE') === 'true',
    auth: {
      user: configService.get<string>('MAIL_USER'),
      pass: configService.get<string>('MAIL_PASS'),
    },
  },
  defaults: {
    from: configService.get<string>('MAIL_FROM'),
  },
}),
}),

```

之后在 AiModule 添加一个 provider:

```

{
  provide: 'SEND_MAIL_TOOL',
  useFactory: (mailerService: MailerService, configService: ConfigService) => {
    const sendMailArgsSchema = z.object({
      to: z
        .email()
        .describe('收件人邮箱地址, 例如: someone@example.com'),
      subject: z.string().describe('邮件主题'),
      text: z.string().optional().describe('纯文本内容, 可选'),
      html: z.string().optional().describe('HTML 内容, 可选'),
    });
  },
}

```

```

return tool(
  async ({to, subject, text, html}: {
    to: string;
    subject: string;
    text?: string;
    html?: string;
  }) => {
    const fallbackFrom =
      configService.get<string>('MAIL_FROM')

    await mailerService.sendMail({
      to,
      subject,
      text: text ?? ' (无文本内容) ',
      html: html ?? `<p>${text ?? ' (无 HTML 内容) '}</p>`,
      from: fallbackFrom,
    });

    return `邮件已发送到 ${to}, 主题为「${subject}」`;
  },
  {
    name: 'send_mail',
    description:
      '发送电子邮件。需要提供收件人邮箱、主题，可选文本内容和 HTML 内容。',
    schema: sendMailArgsSchema,
  },
);
inject: [MailerService, ConfigService],
},

```

绑定一下：

直接调用会渲染默认 tool call 组件

我们再加一个单独的组件用于渲染发送邮件的 tool

试下效果：

神光的编程秘籍

完成。

整体总结下：

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

我们基于 AGUI 协议实现了流式渲染文本、tool call 组件的效果。

用的是 Vercel AI SDK 的 Data Stream Protocol。

后端用 LangChain 来写 Agent，我们不再手写 agent loop，直接用了 createAgent 的 api

通过 @ai-sdk/langchain 把 stream 转为基于 Data Stream Protocol 协议的 SSE 流

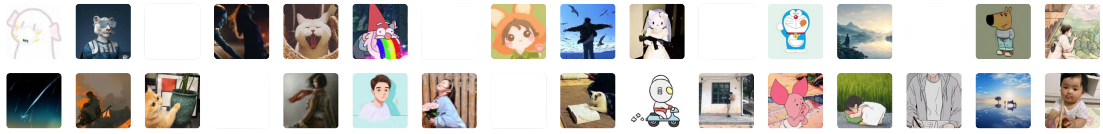
前端用 @ai-sdk/react 或者 @ai-sdk/vue 的 useChat 来解析这个 SSE 流，拿到 messages。

根据 message 是文本还是 tool call 做不同的渲染

文本用 streamdown 流式渲染，会解析 markdown 的表格、mermaid 流程图、代码等语法，用不同组件展示

tool call 则是自定义组件实现渲染。

对接了 AGUI 协议后，Agent 的交互体验就好很多了。



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇

给 Agent 加上语音交互：ASR + 流式 TTS

下一篇

图编排引擎：LangGraph 和多 Agent 架构

